CI/CD Pipeline for Brain Tasks Application using AWS Services

Aim

To build and deploy a containerized Brain Tasks application using AWS CodeBuild, Amazon ECR, Amazon EKS, Kubernetes, Docker, and CloudWatch.

Tools Used

- GitHub
- Docker
- AWS CodeBuild
- Amazon ECR
- Amazon EKS
- Kubernetes
- CloudWatch
- kubectl

Architecture Diagram

GitHub Repository

↓

AWS CodePipeline

↓

AWS CodeBuild

↓

Docker Image Build

↓

Amazon ECR

↓

Amazon EKS

↓

Kubernetes Deployment

↓

LoadBalancer Service

↓

Brain Tasks Application

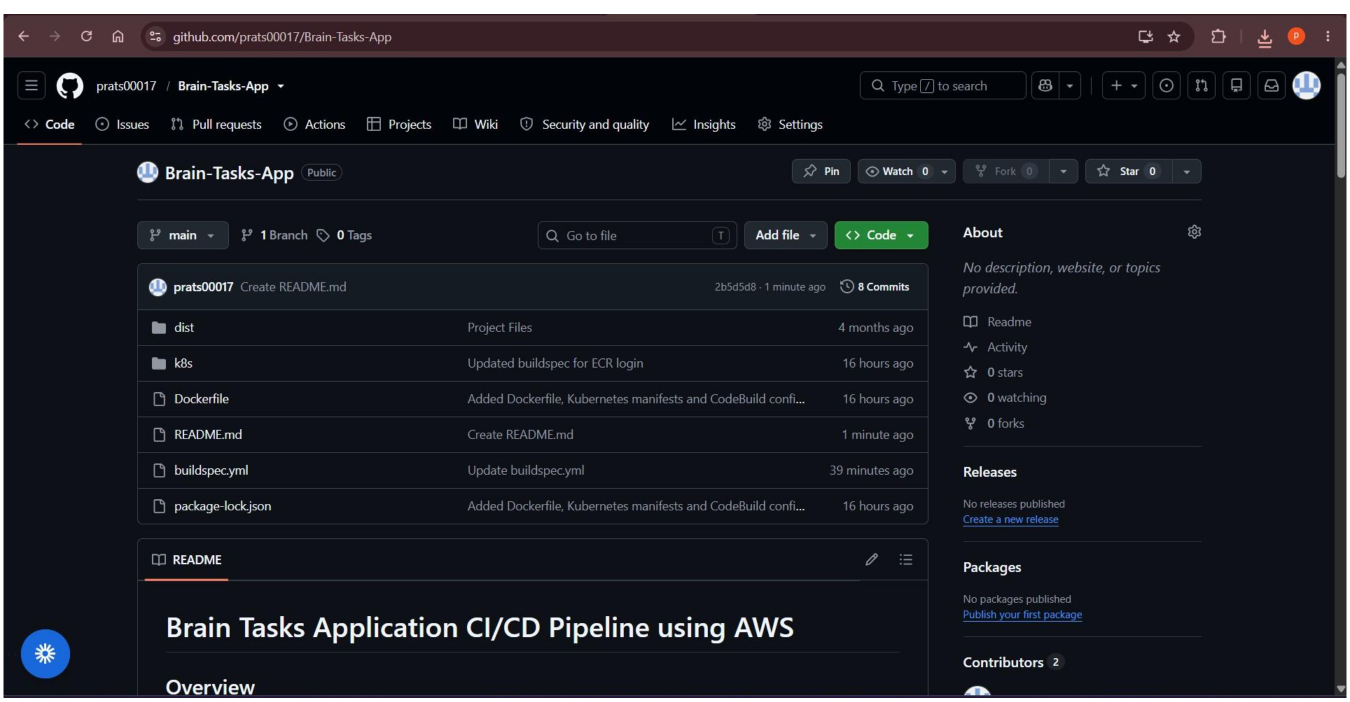
↓

CloudWatch Monitoring

Step 1: Source Code Repository

The Brain Tasks application source code was maintained in GitHub.

Screenshot:



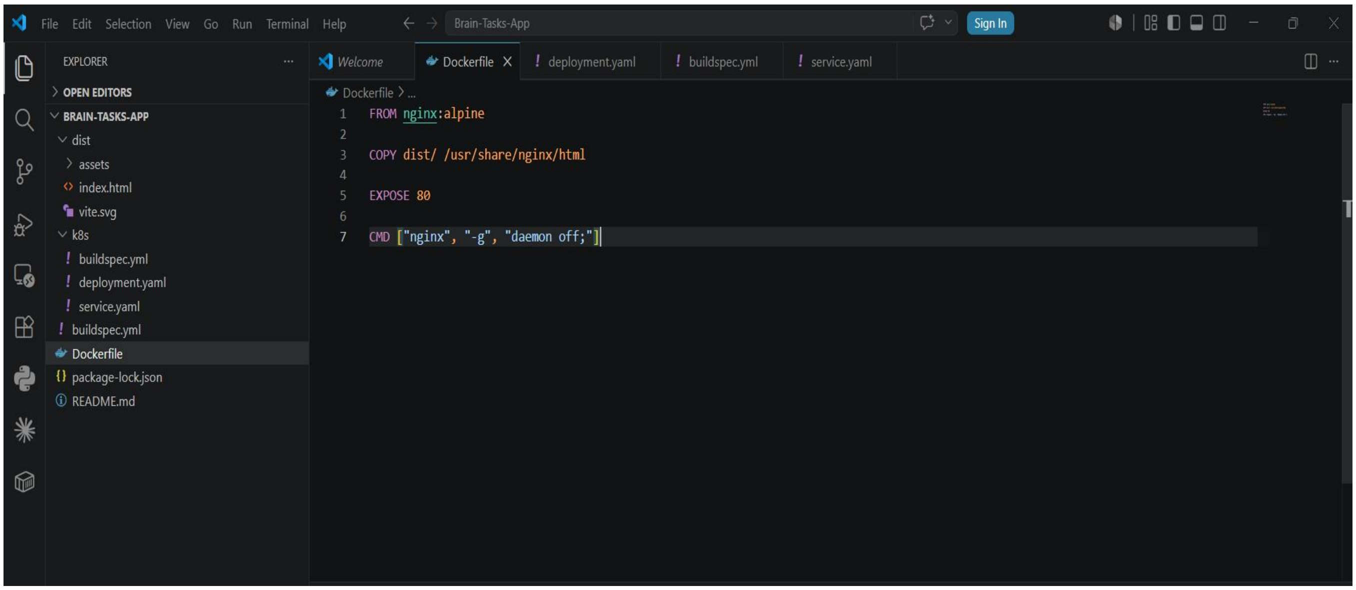
Step 2: Dockerization

A Dockerfile was created to containerize the application.

Command:

```
docker build -t brain-tasks-app .
```

Screenshot:



Step 3: AWS CodeBuild

Created CodeBuild project:

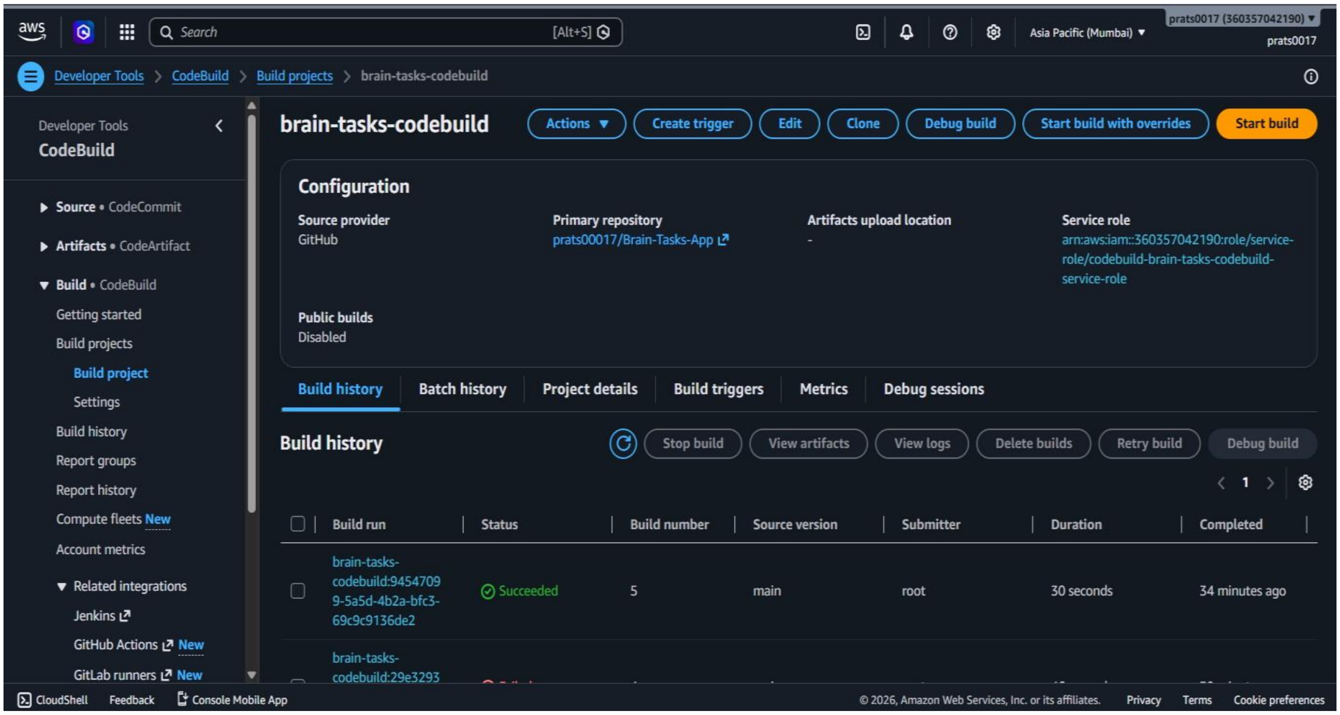
brain-tasks-codebuild

Configured:

- Source repository integration with GitHub.
- buildspec.yml file for automated build instructions.
- Docker image build process.
- Integration with AWS CodePipeline.

Build execution completed successfully and artifacts were generated.

Screenshot:



Step 4: AWS CodePipeline

Created pipeline:

brain-tasks-pipeline

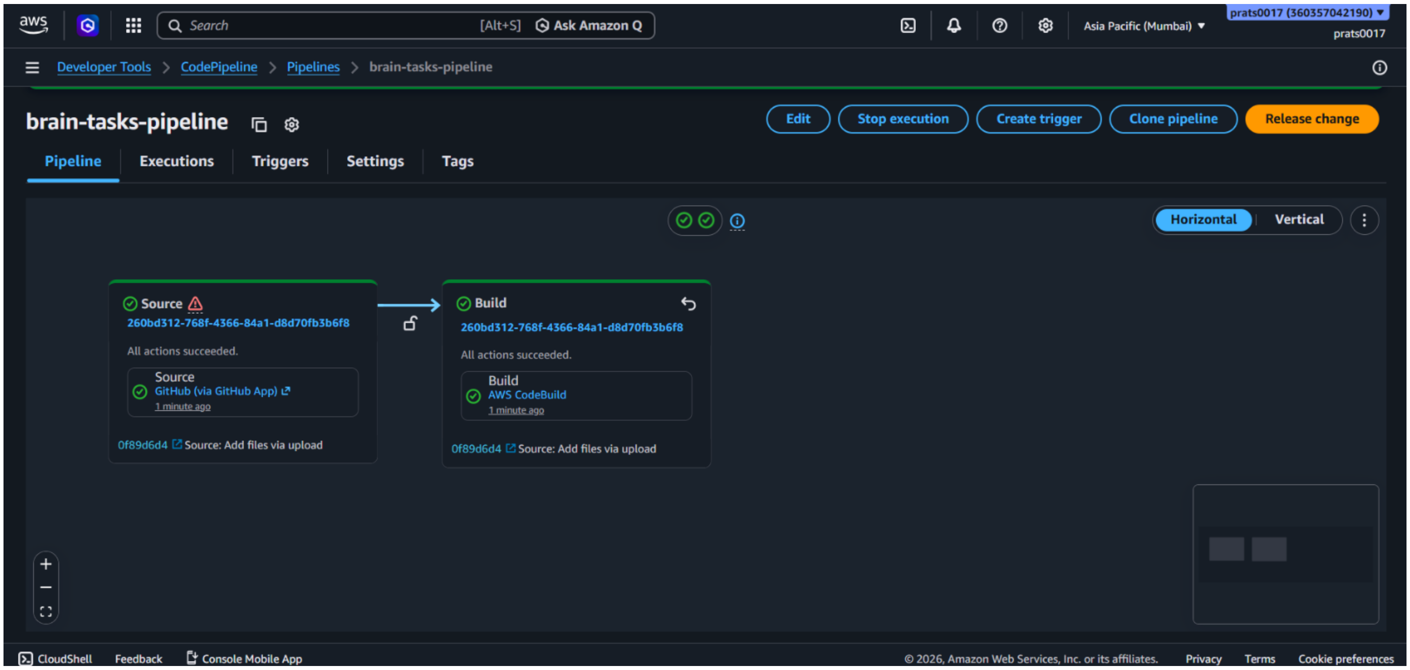
Configured stages:

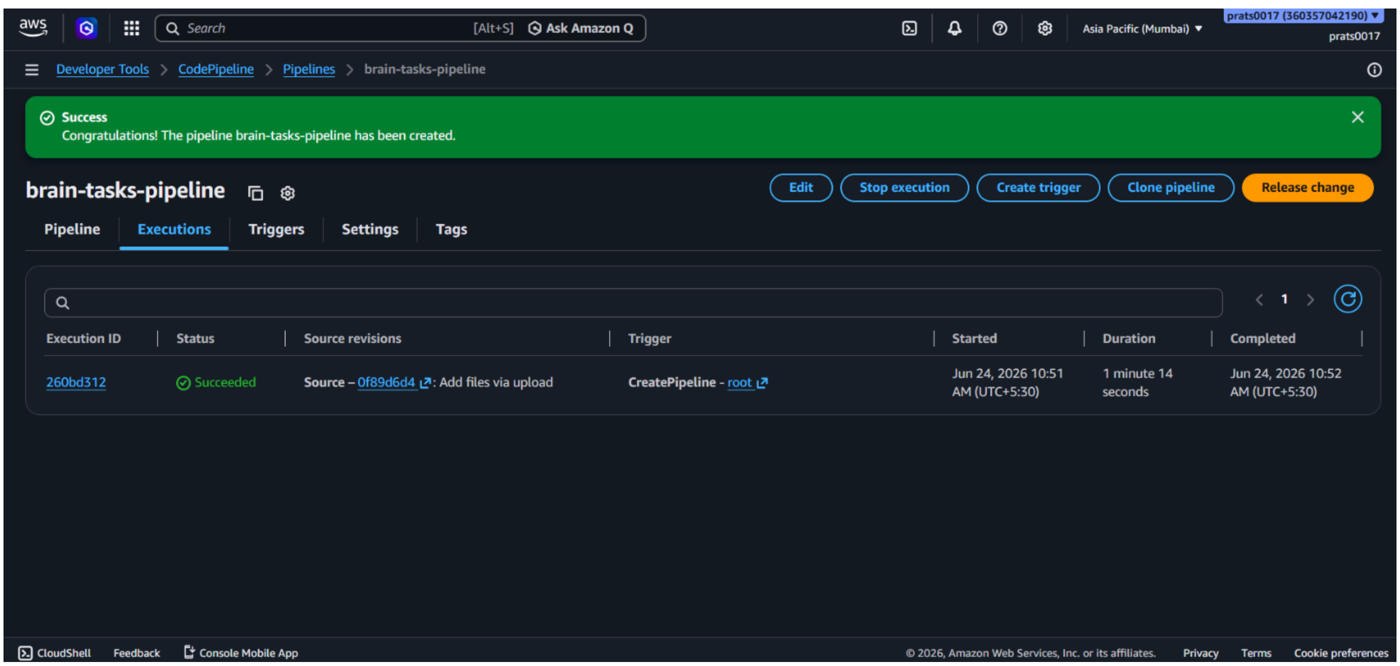
1. Source Stage
   - GitHub Repository
2. Build Stage
   - AWS CodeBuild

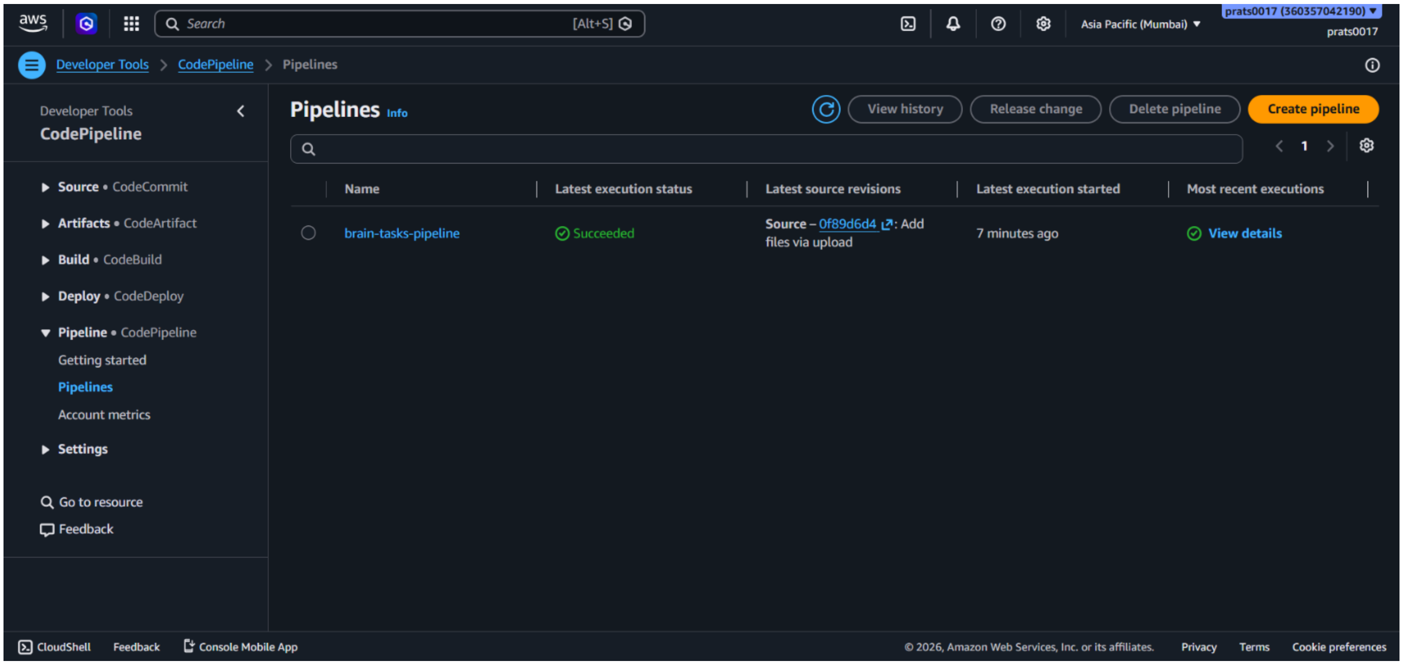
Pipeline execution was completed successfully.

This automated the Continuous Integration process and ensured that source code changes trigger the build workflow.

Screenshots:







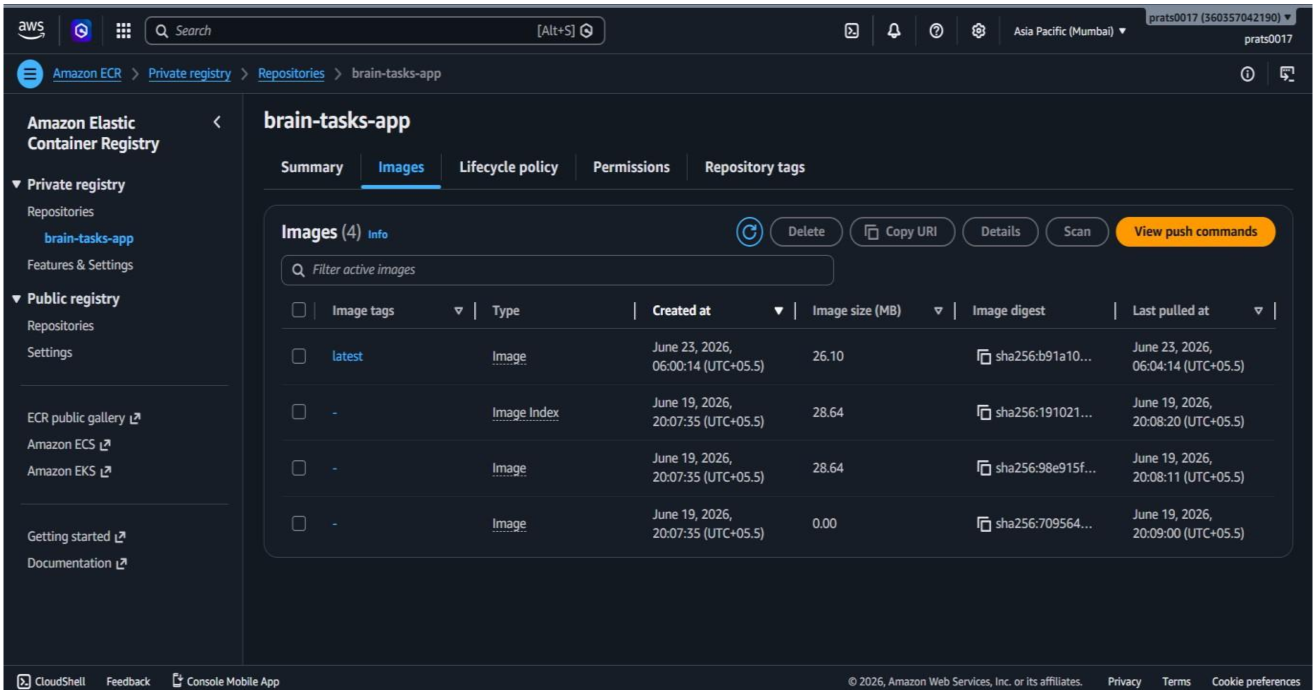
Step 5: Amazon ECR

Repository:

brain-tasks-app

Docker image was pushed successfully.

Screenshot:



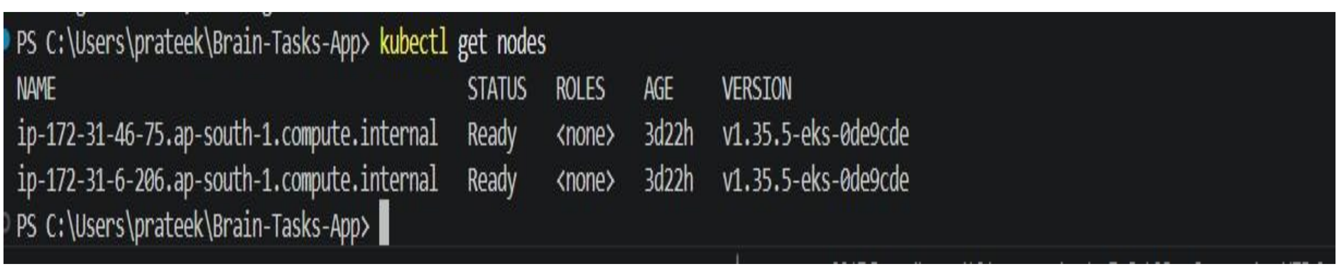
Step 6: Amazon EKS

Cluster nodes verified.

Command:

```
kubectl get nodes
```

Screenshot:



Step 7: Deployment

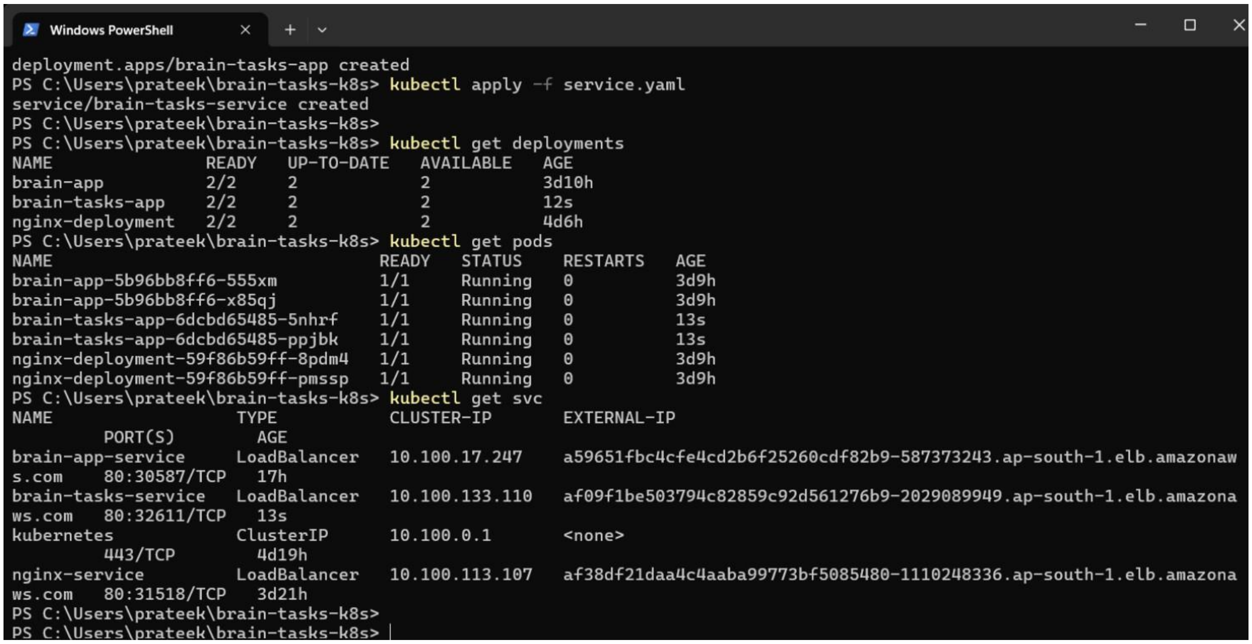
Applied Kubernetes manifests.

Commands:

```
kubectl apply -f deployment.yaml
```

```
kubectl apply -f service.yaml
```

Screenshot:



Step 8: Pods Verification

Command:

```
kubectl get pods
```

Screenshot:

As above

Step 9: Services Verification

Command:

```
kubectl get svc
```

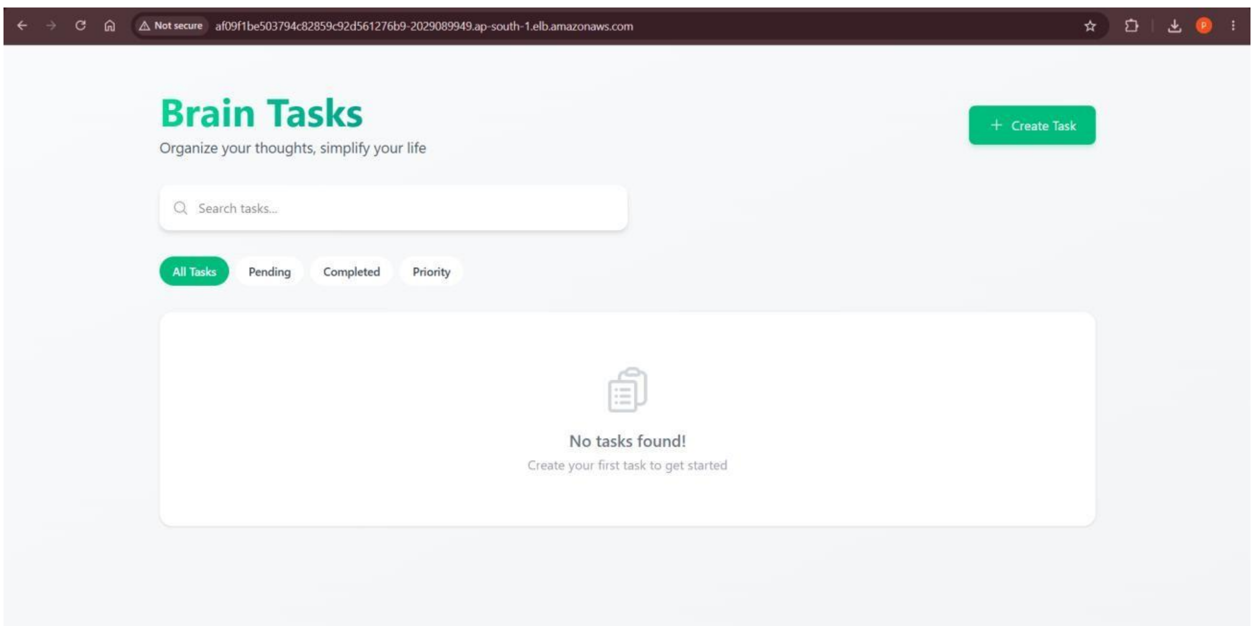
Screenshot:

As above

Step 10: Application Output

Application was accessed through the external LoadBalancer URL.

Screenshot:



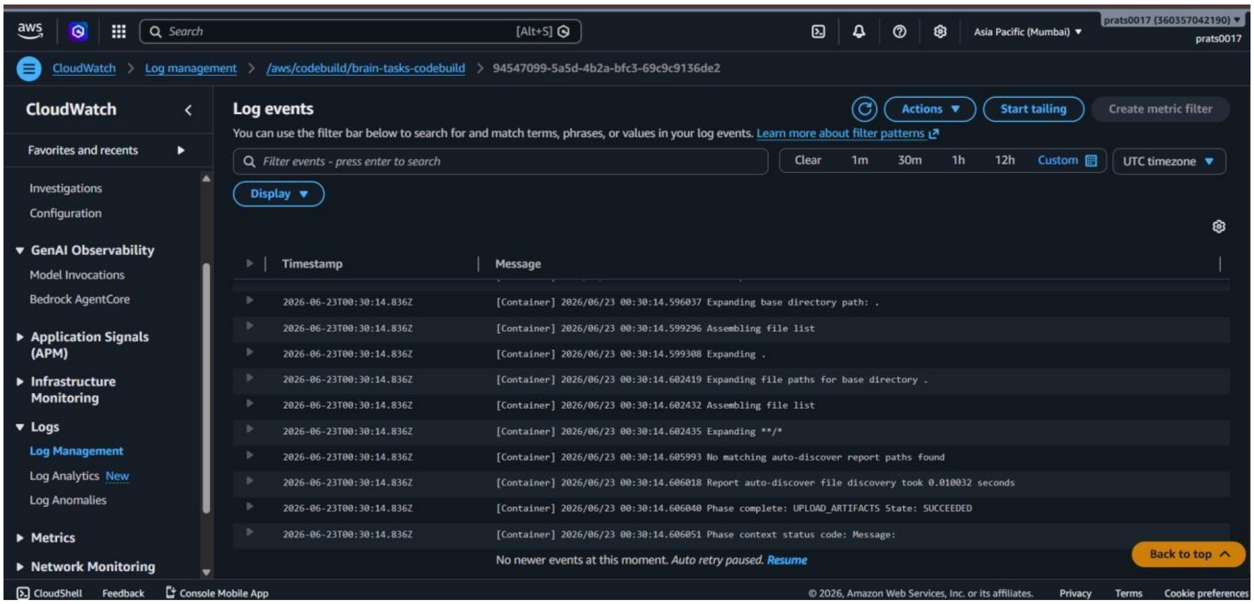
Step 11: CloudWatch Monitoring

CloudWatch log group:

/aws/codebuild/brain-tasks-codebuild

Build logs were monitored successfully.

Screenshot:



Commands Used

```
kubectl get nodes
kubectl get deployments
kubectl get pods
kubectl get svc
kubectl apply -f deployment.yaml
kubectl apply -f service.yaml
```

Result

Successfully implemented and deployed the Brain Tasks application using a complete CI/CD pipeline with GitHub, AWS CodePipeline, AWS CodeBuild, Amazon ECR, Amazon EKS, Kubernetes and CloudWatch.

The application was containerized using Docker, images were stored in Amazon ECR, deployed on Amazon EKS, and exposed externally through a LoadBalancer service. Monitoring and build activities were tracked using CloudWatch.

Conclusion

The project demonstrates an end-to-end DevOps workflow using AWS cloud-native services. Continuous Integration was achieved through AWS CodePipeline and AWS CodeBuild, while Continuous Deployment was performed using Kubernetes on Amazon EKS.

The Brain Tasks application was successfully deployed and made accessible through a LoadBalancer service. Monitoring and logging were achieved using Amazon CloudWatch, providing a complete CI/CD implementation.